

UNIVERSIDAD INTERNACIONAL DEL ECUADOR



CARRERA DE INGENIERÍA EN CIBERSEGURIDAD

LÓGICA DE PROGRAMACIÓN 1-CIB-1A

PROYECTO FINAL

TEMA:

PRESENTACIÓN COMPLETA DEL SOFTWARE SELECCIONADO (SNAKE)

AUTOR: ANDRADE MEDINA RONALD FABRICIO

FECHA DE ENTREGA: 28/06/2026

2026-2027

TRABAJO FINAL

INTRODUCCIÓN

El presente proyecto consiste en el desarrollo del clásico videojuego de la serpiente (Snake) generando los diagramas de flujo y funcionalidad, y para el código utilizando el lenguaje de programación Python y la biblioteca gráfica turtle. El objetivo principal es aplicar los conocimientos adquiridos en la asignatura para la construcción de este.

Antes de mencionar en qué consiste el juego Snake, debemos hablar de su origen y evolución. El juego de la Serpiente se originó en las máquinas recreativas bajo el nombre de "Blockade", de Gremlin Industries, y llegó a los hogares en 1978 como "Worm" para el microordenador TRS-80. En el ámbito móvil, aunque el Nokia 3310 del año 2000 es el más emblemático, fue realmente el Nokia 6110 el que introdujo el juego por primera vez en 1997. Hoy en día, el juego sigue vigente a través de versiones modernas en teléfonos Nokia, como la colorida "Snake Xenzia" o variantes de aventura con niveles más grandes, y también mediante Google, que ofrece una versión de transporte público en Google Maps y otra variante altamente personalizable con múltiples modos y ajustes directamente desde su buscador. (HMD, 2023)

El juego consiste en controlar una serpiente que se desplaza por una cuadrícula, con el fin de comer alimentos (representados por círculos rojos) que aparecen aleatoriamente. Cada vez que la serpiente come, aumenta su longitud y el puntaje del jugador. La partida termina si la serpiente choca contra las paredes o contra su propio cuerpo, mostrando un mensaje de derrota. También se contempla una condición de victoria: si la serpiente logra ocupar todas las celdas disponibles del tablero, el jugador gana la partida.

Este proyecto no solo demuestra la capacidad de construir un programa funcional a partir de elementos básicos del lenguaje, sino que también aborda problemas reales de la programación de videojuegos, como la detección de colisiones, la gestión del estado del juego y la respuesta a eventos de teclado en tiempo real.

1. DESCRIPCIÓN DEL PROBLEMA

1.1. Contexto

Se requiere un programa en Python que simule el juego Snake con las siguientes características: la serpiente se mueve en una cuadrícula 2D, controlada por las teclas W (arriba), A (izquierda), S (abajo) y D (derecha). Una manzana (círculo rojo) aparece en posiciones aleatorias de la cuadrícula. Al comer la manzana, la serpiente aumenta su longitud en un segmento y el puntaje se incrementa en 10 unidades. El juego termina si la cabeza de la serpiente colisiona con los bordes del área de juego o con cualquier parte de su cuerpo. Si la serpiente logra llenar completamente el tablero (es decir, no quedan celdas libres para colocar una nueva manzana), el jugador gana la partida.

1.2. Requisitos funcionales

Los requisitos funcionales son especificaciones detalladas que definen exactamente lo que un sistema debe hacer para cumplir su propósito, abarcando sus acciones, tareas, interacciones con los usuarios y las respuestas a diversas entradas o eventos. En el ámbito del software, establecer estos requisitos es fundamental para asegurar que el producto final cumpla con las expectativas de las partes interesadas y satisfaga las necesidades previstas del negocio. (Valera, 2026)

Requisitos planteados en el proyecto:

- La serpiente debe poder moverse en cuatro direcciones.
- El cambio de dirección debe impedir giros de 180° que causarían autocolisión instantánea.
- La comida debe generarse en posiciones alineadas con la cuadrícula (múltiplos de 20 píxeles) y nunca sobre el cuerpo de la serpiente, es decir se debe generar por donde la serpiente pase y no entre las líneas de cada celda.
- Al comer, la serpiente debe crecer un segmento, que se añade en la cola.

- El puntaje debe mostrarse permanentemente en pantalla y actualizarse en tiempo real.
- Al chocar con una pared o con el cuerpo, se debe mostrar un mensaje de derrota con el puntaje obtenido y reiniciar automáticamente la partida.
- Si el tablero se llena por completo, se debe mostrar un mensaje de victoria con el puntaje final y detener el juego.
- El juego debe responder correctamente, aunque el usuario pulse varias teclas en rápida sucesión.
- La velocidad del juego debe ser constante y ajustable (controlada por time.sleep).

2. RELACIÓN CON LA ASIGNATURA

El desarrollo del juego de la serpiente nos ha permitido aplicar los conceptos básicos del curso de una manera práctica. No es solo una serie de ejercicios, sino que requiere combinar diferentes estructuras de datos y técnicas de programación para crear un sistema interactivo completo.

Uno de los aspectos clave del programa es el uso de variables de diferentes tipos. La puntuación se almacena en un número entero, la dirección de la serpiente en una cadena y las banderas de control en valores booleanos. Esto refleja la necesidad de elegir el tipo de dato adecuado para cada aspecto del juego.

Para gestionar el cuerpo de la serpiente, se utiliza una lista que almacena cada segmento. Las listas permiten agregar elementos dinámicamente, recorrerlos en secuencia y vaciarlos cuando se reinicia el juego. Esto es ideal porque el orden de los segmentos es importante.

La decisión de utilizar tuplas y conjuntos para controlar las celdas ocupadas es importante. Cada celda se identifica con sus coordenadas, que se almacenan como tuplas. Las tuplas son inmutables y pueden ser elementos de un conjunto. Esto permite verificar rápidamente si una celda está libre para colocar comida.

Las estructuras de control condicionales, como if y elif, se utilizan en todo el código para decidir si la nueva dirección es válida, si la serpiente ha chocado con una pared o si se debe crecer. Los bucles for se utilizan para dibujar líneas, mover segmentos y buscar celdas libres.

La modularización mediante funciones ha sido clave para la legibilidad y el mantenimiento del código. Cada función se encarga de un aspecto específico del comportamiento, lo que evita la repetición de código y facilita la depuración y ampliación del programa.

El manejo de eventos se implementa mediante la escucha de teclado, lo que permite al programa responder a las acciones del usuario. El uso de bibliotecas externas, como turtle y random, demuestra la capacidad de integrar módulos de la biblioteca estándar de Python.

En resumen, el desarrollo de este juego ha permitido aplicar de manera práctica los conceptos fundamentales del curso, demostrando la comprensión de estructuras de datos, técnicas de programación y principios de diseño de software.

3. EXPLICACIÓN DEL SISTEMA DESARROLLADO

El sistema se organiza en una única capa. La lógica del juego se maneja en un script Python que ejecuta un bucle principal. No se utiliza programación orientada a objetos; en su lugar, se emplea un enfoque procedimental basado en funciones y variables globales.

Los módulos lógicos identificados son:

- Inicialización: Configura la ventana, dibuja la cuadrícula y crea los turtles.
- Gestión de entrada: Funciones de teclado que modifican una variable buffer (nueva_direccion).
- Lógica del juego (bucle principal):
 - ✓ Actualización de pantalla.
 - ✓ Verificación de colisiones (paredes, cuerpo).

- ✓ Procesamiento de la dirección (buffer → dirección real).
 - ✓ Movimiento de la cabeza.
 - ✓ Movimiento del cuerpo (arrastre).
 - ✓ Gestión del crecimiento (bandera crecer y conjunto ocupadas).
 - ✓ Detección de comida (consumo, puntaje, nueva comida, condición de victoria).
 - ✓ Control de velocidad (time.sleep).
- Utilidades: Funciones de mensajes, reinicio, generación de comida segura.

3.1. Datos clave

- **cabeza:** turtle que representa la cabeza; tiene un atributo dirección (string) añadido dinámicamente.
- **nueva_direccion:** string que almacena la última dirección solicitada por el jugador. Actúa como buffer para desacoplar las pulsaciones del movimiento real.
- **segmentos:** lista de turtles (cuerpo). El orden importa: segmentos[0] es el más cercano a la cabeza.
- **ocupadas:** conjunto de tuplas (x, y) con todas las celdas ocupadas. Se actualiza en cada movimiento.
- **crecer:** booleano que indica si en el próximo movimiento debe añadirse un segmento extra.
- **puntaje:** entero que acumula 10 puntos por comida.

3.2. Configuración Ventana

La ventana se configura con `turtle.Screen()`, `bgcolor("black")`, `setup(600,600)` y `tracer(0)`. La cuadrícula se dibuja con un turtle oculto que recorre x desde -290 hasta 290 en pasos de 20 para líneas verticales, e y en el

mismo rango para horizontales. El color "gray20" y pensize(1) proporcionan una cuadrícula sutil. La cabeza se sitúa en (0,0), la comida en (0,100).

Turtle Graphics es un módulo de Python que permite dibujar y animar mediante el control de un cursor o "tortuga" virtual. Al darle comandos de movimiento y giro, la tortuga se desplaza por la pantalla dejando un trazo para crear desde líneas hasta formas elaboradas en tiempo real. (Jeremiah, 2026)

3.3. Control de dirección

Las cuatro funciones `ir_arriba`, `ir_abajo`, `ir_izq`, `ir_der` modifican `nueva_direccion` solo si la dirección opuesta a la actual real (`cabeza.direccion`) no es la solicitada. Esto evita cambios a 180° incluso si se pulsan varias teclas rápido, es decir permite que no tengamos una autocolisión.

3.4. Movimiento de la cabeza y el cuerpo

El crecimiento de la serpiente se implementa mediante una bandera booleana denominada `crecer` que se activa al detectar la colisión con la comida. En la siguiente iteración del bucle principal, antes de actualizar el conjunto de posiciones ocupadas, se evalúa el valor de esta bandera. Si `crecer` es `True`, se invoca la función `agregar_segmento(ant_x, ant_y)`, la cual crea una nueva instancia de Turtle con forma cuadrada y color verde oscuro. Esta nueva instancia se posiciona en la coordenada que corresponde a la cola actual de la serpiente —es decir, la última posición del último segmento si existe, o la posición anterior de la cabeza en caso contrario—. Dado que no se descarta ninguna celda del conjunto ocupadas, la serpiente incrementa su longitud en una unidad sin que se produzca un desplazamiento neto de la cola. Por el contrario, si `crecer` es `False`, se procede a eliminar del conjunto ocupadas la celda correspondiente a la posición más alejada de la cabeza, simulando así el avance convencional del cuerpo sin modificar su tamaño.

3.5. Generación de comida y detección de colisión

La función `nueva_comida()` genera coordenadas aleatorias $x = \text{random.randint}(-14,14)*20$, $y = \text{random.randint}(-14,14)*20$. Si esa tupla no pertenece a ocupadas, se coloca la comida allí y retorna True. Si tras 1000 intentos no encuentra sitio, retorna False. Al detectar $\text{cabeza.distance(comida)} < 20$, se llama a `nueva_comida()`. Si retorna False, se declara victoria: se suma el último puntaje, se muestra mensaje y se rompe el bucle con `break`. Si retorna True, se suma puntaje, se actualiza marcador y se pone `crecer = True`.

La colisión con el cuerpo se evalúa recorriendo segmentos y viendo si $\text{cabeza.distance(seg)} < 20$. En caso positivo, se muestra derrota y se reinicia.

3.6. Reinicio del juego

La función `reiniciar_juego()` restaura las variables a su estado inicial: cabeza a (0,0), dirección "stop", `nueva_direccion="stop"`, elimina todos los segmentos con `seg.reset()` y `hideturtle()`, limpia la lista, resetea puntaje, marcador, ocupadas (añadiendo (0,0)) y `crecer = False`.

3.7. Mensaje al jugador

`mostrar_mensaje(texto, duracion)` usa un turtle dedicado. Limpia, escribe con `write()`, llama a `ventana.update()` (clave para que se vea antes de la pausa), duerme la duración indicada y limpia de nuevo. Así se muestran los mensajes de “¡Has perdido!” y “¡Has ganado! Llenaste el tablero.” con el puntaje.

4. PROBLEMAS DURANTE LA ELABORACIÓN DEL CÓDIGO

4.1. Muerte instantánea al comer la primera comida

Al comenzar una partida y dirigir la serpiente hacia la primera manzana, en el momento de comerla, la serpiente moría sin que apareciera ningún mensaje en la pantalla. El juego simplemente se reiniciaba o se detenía, sin dar información al jugador.

Causado porque la bandera “crecer” se activaba al comer. En el siguiente movimiento, se añadía un nuevo segmento al cuerpo mediante la función “agregar_segmento”. Originalmente, esta función creaba el nuevo cuadrado en la posición de la cabeza o en (0,0). Si no existían segmentos previos, el nuevo segmento se situaba exactamente donde estaba la cabeza en ese momento. Como la cabeza ya se había movido a la celda de la comida, el nuevo segmento se colocaba allí, solapándose con la cabeza instantáneamente. La verificación de colisión con el cuerpo detectaba esa superposición y provocaba la muerte sin que el jugador pudiera ver qué ocurría. El mensaje no se visualizaba porque la función “mostrar_mensaje” no forzaba un “ventana.update” antes de la pausa, por lo que el texto no llegaba a renderizarse.

Solución: Se rediseñó la lógica de crecimiento. La función “agregar_segmento” ahora recibe la posición anterior de la cabeza. Si no hay cuerpo, coloca el nuevo segmento ahí, no en la nueva posición de la cabeza. Se movió el bloque de crecimiento justo después de mover la cabeza y antes de descartar la cola. Así, el nuevo segmento ocupa la celda que la cabeza acaba de abandonar, que ya no está ocupada por ella. Se añadió “ventana.update” dentro de “mostrar_mensaje” para garantizar que cualquier mensaje se muestre inmediatamente.

4.2. La serpiente se desplazaba sobre las líneas de la cuadrícula

El fondo tenía líneas dibujadas para simular celdas, pero la serpiente, en lugar de moverse por el interior de cada celda, avanzaba exactamente sobre las líneas. Esto daba una sensación visual extraña, como si la serpiente caminara por los bordes de las baldosas en vez de por el centro.

Causado porque el dibujo original trazaba líneas en $x = -280, -260, \dots, 280$ y en y con los mismos valores. La cabeza de la serpiente se desplaza precisamente en pasos de 20 por esas mismas coordenadas. Coincidían las posiciones de movimiento con las líneas.

Solución: Se desplazaron las líneas 10 píxeles hacia fuera del área de movimiento. Las verticales se colocaron en $x = -290, -270, \dots, 290$ y las horizontales en $y = -290, -270, \dots, 290$. De este modo, las posiciones $-280, -260$, etc., quedan justo en el centro de cada celda, logrando el efecto visual deseado de una serpiente que ocupa los cuadros.

4.3. Bug del giro rápido que terminaba en autocolisión

Cuando el jugador presionaba dos teclas de dirección muy rápidas que en combinación equivalían a un giro de 180° , la serpiente giraba sobre sí misma y moría al chocar con su propio cuerpo, aunque la intención del jugador fuera, por ejemplo, ir hacia abajo, luego a la izquierda, y luego arriba. El resultado era un giro directo de abajo a arriba.

Este error fue causado porque las funciones asociadas a las teclas modificaban directamente la dirección de la cabeza tras verificar que la nueva dirección no fuera opuesta a la dirección actual en ese instante. Al presionar izquierda, la dirección pasaba de “down” a “left”. Inmediatamente después, al presionar arriba, se comparaba “up” con “left”, no siendo opuestos, y se permitía el cambio. Así, en un solo fotograma la dirección pasaba de “down” a “left” y luego a “up”, ejecutando un giro prohibido.

Solución: Se introdujo una variable intermedia “nueva_direccion” que actúa como buffer. Las funciones de teclado solo actualizan esta variable, verificando contra la dirección real del último movimiento. En el bucle principal, una única vez por fotograma, se evalúa si “nueva_direccion” es válida con respecto a la dirección real y, de ser así, se actualiza. Si se pulsaron varias teclas, solo la última antes de la evaluación queda registrada, pero siempre se compara contra la dirección real, no contra valores intermedios no ejecutados. Esto elimina por completo el falso giro de 180° .

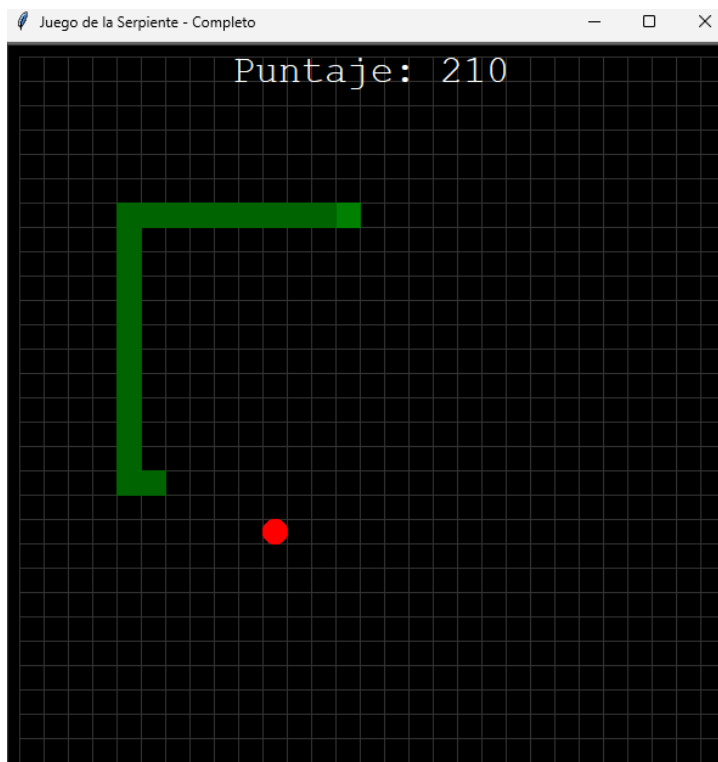
5. Reflexión sobre el impacto de la tecnología

Hoy en día, la tecnología es un pilar fundamental de nuestra vida cotidiana. Al observar nuestro entorno, resulta fascinante ver cómo nos ayuda a resolver problemas complejos con facilidad. Detrás de cada pantalla y mensaje hay un enorme esfuerzo por hacer la vida más accesible y mantenernos conectados. Este desarrollo ha transformado radicalmente la comunicación humana. Tras un sitio web o un mensaje enviado opera una infraestructura masiva que garantiza el flujo seguro de la información. Comprender este mecanismo revela que las herramientas digitales no nos aíslan; por el contrario, fomentan la colaboración y el progreso colectivo. Nuestra interacción constante con los sistemas digitales también está transformando nuestra forma de pensar. Ahora buscamos soluciones más eficientes, automatizamos tareas y desarrollamos una mentalidad más analítica y organizada. Esta lógica no se limita a las computadoras; la aplicamos a diario al planificar nuestro tiempo o gestionar el tráfico en las grandes ciudades. Cuando la tecnología falla, naturalmente recordamos que es una creación humana y, por tanto, imperfecta. Afrontar estos desafíos técnicos cultiva la paciencia y nos enseña que el progreso sostenible requiere esfuerzo, resiliencia y mejora continua. Además, comprender el funcionamiento de las plataformas que utilizamos nos convierte en usuarios más críticos y perspicaces. Ya no aceptamos el contenido de forma pasiva; en su lugar, cuestionamos el impacto de lo que vemos en pantalla, tomando así el control de nuestras vidas digitales. Por último, los avances tecnológicos conllevan la responsabilidad de construir un entorno digital justo y seguro. Debemos priorizar el diseño de sistemas que respeten la privacidad, la transparencia y la honestidad. El objetivo final debe ser que la tecnología sirva como motor de igualdad, oportunidades y protección para todos.

6. Conclusión

El juego de la serpiente (Snake) se implementó en Python utilizando la biblioteca “turtle” y cumple con todos los requisitos funcionales y no funcionales establecidos. El producto final es un programa interactivo, visualmente claro y robusto, libre de los errores que surgieron durante el desarrollo. Este proceso demostró la importancia de diseñar una arquitectura clara, aunque fuera procedimental, para facilitar la depuración, así como de utilizar estructuras de datos adecuadas, tales como listas para el cuerpo de la serpiente, conjuntos para las celdas ocupadas y tuplas para las coordenadas. También puso de relieve la necesidad de gestionar los eventos de entrada mediante un búfer para evitar comportamientos inesperados, además de documentar y probar sistemáticamente todas las funcionalidades. Los desafíos encontrados —en particular, los cambios bruscos de dirección y la autocolisión al alimentarse— sirvieron como oportunidades de aprendizaje que reforzaron la comprensión del flujo del programa y la necesidad de mantener la coherencia entre el estado visual y el modelo de datos.

Anexos



Referencias

HMD. (17 de Julio de 2023). *El juego de la serpiente revisitado: datos sorprendentes y curiosidades fascinantes para el Día Mundial de la Serpiente*. Obtenido de HMD: https://www.hmd.com/en_gb/blog/snake-game-revisited-surprising-facts-and-fascinating-trivia

Jeremiah, O. (18 de Mayo de 2026). *Python Turtle Graphics: una forma divertida de aprender los fundamentos*. Obtenido de datacamp: <https://www.datacamp.com/es/tutorial/turtle-graphics>

Valera, F. (24 de Abril de 2026). *La guía definitiva sobre gestión de requisitos y ALM*. Obtenido de v/sure: <https://visuresolutions.com/es/gu%C3%ADa-de-limosna/requisitos-funcionales-vs-no-funcionales/>